

BASH SCRIPTING FUNDAMENTALS

Projects

Build real automation scripts end to end

Merit Advisory

Bash Scripting Fundamentals Bootcamp

Beginner-friendly lecture materials

July 2026

July 2026



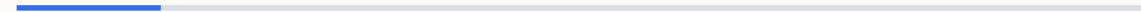
Agenda

- 1 Section 1: Project habits before coding — 7 topics
- 2 Section 2: Project A sysinfo report basics — 7 topics
- 3 Section 3: Project A sysinfo report details — 7 topics
- 4 Section 4: Project B backup-dir basics — 7 topics
- 5 Section 5: Project B backup-dir polish — 7 topics
- 6 Section 6: Project C log scanner basics — 7 topics
- 7 Section 7: Project C log scanner details — 7 topics
- 8 Section 8: Project D toolbelt CLI basics — 7 topics

- 9 Section 9: Project D toolbelt polish — 7 topics
- 1 Section 10: Documentation and safety — 7 topics
- 0
- 1 Section 11: Build plan and acceptance — 7 topics
- 1
- 1 Section 12: Common bugs and portfolio polish — 7 topics
- 2

1 Section 1

Project habits before coding



1.1 Start with one sentence requirements

- A project begins with what the script must do

Requirement

```
echo 'Create a sysinfo report file'
```

- Keep the first version small enough to finish

Scope line

```
echo 'Input: none, output: report.txt'
```

1.2 Name scripts after actions

- Clear names make command history readable

Project names

```
sysinfo-report  
backup-dir
```

- Use lowercase words separated by hyphens

Tool name

```
log-scan
```

1.3 Create a small project folder

- Keep scripts, tests, and samples together

Make folders

```
mkdir -p project/bin project/tests
```

- A predictable layout helps demos

Samples folder

```
mkdir -p project/samples
```

1.4 Use a Bash shebang

- The shebang tells the OS how to run the script

Shebang

```
#!/usr/bin/env bash
```

- env finds bash through PATH

Run file

```
./bin/sysinfo-report
```


1.5 Use strict mode in projects

- Project scripts should fail predictably

Strict header

```
set -euo pipefail
```

- Strict mode catches missing variables and failed commands

Expected no match

```
grep -q ERROR log || true
```

1.6 Make scripts executable

- chmod allows direct script execution

Add execute bit

```
chmod +x bin/sysinfo-report
```

- Version control can remember executable bits

Run directly

```
./bin/sysinfo-report
```

1.7 Commit after each working slice

- Small commits capture progress safely

Check status

```
git status --short
```

- Each slice should run before committing

Commit slice

```
git add . && git commit -m 'add sysinfo header'
```

2 Section 2

Project A sysinfo report basics

2.1 Sysinfo report requirements

- The script writes a readable machine report

Requirement note

```
echo 'sysinfo-report writes report.txt'
```

- It should include host, kernel, uptime, memory, and disk

Output name

```
report=system-report.txt
```

2.2 Choose the report file

- A variable makes the output path easy to change

Default report

```
report="system-report.txt"
```

- Quote the report path on every use

Create empty report

```
: > "$report"
```

2.3 Write a report title

- A title makes the report self-explanatory

Title line

```
echo 'System Information Report' > "$report"
```

- Redirect once to create the file

Separator

```
echo '===== ' >> "$report"
```

2.4 Add a timestamp

- Reports should say when they were produced

Timestamp value

```
now=$(date '+%Y-%m-%d %H:%M:%S')
```

- Use one consistent date format

Write timestamp

```
echo "Generated: $now" >> "$report"
```


2.5 Collect the hostname

- hostname identifies the machine

Hostname

```
host=$(hostname)
```

- Store command output before writing

Report hostname

```
echo "Hostname: $host" >> "$report"
```

2.6 Collect kernel information

- `uname` shows operating system and kernel details

Kernel value

```
kernel=$(uname -a)
```

- `uname -a` is verbose and useful for diagnostics

Report kernel

```
echo "Kernel: $kernel" >> "$report"
```

2.7 Collect uptime

- uptime shows load and running time

Uptime value

```
up=$(uptime)
```

- The output is already human-readable

Report uptime

```
echo "Uptime: $up" >> "$report"
```

3 Section 3

Project A sysinfo report details

3.1 Add disk usage with df

- df -h reports human-readable disk usage

Disk heading

```
echo 'Disk usage:' >> "$report"
```

- The root filesystem is a useful first target

Disk command

```
df -h / >> "$report"
```

3.2 Add memory with free

- free -h reports memory in readable units

Memory heading

```
echo 'Memory:' >> "$report"
```

- Not every minimal system has free installed

Memory command

```
free -h >> "$report"
```

3.3 Format sections with blank lines

- Blank lines make command output easier to scan

Blank line

```
echo >> "$report"
```

- Use a tiny helper when repeated

Section helper

```
section() { echo; echo "$1"; } >> "$report"
```

3.4 Group report writing

- A brace group can share one redirection

Grouped output

```
{ echo 'Report'; date; } > "$report"
```

- This reduces repeated append operators

Append group

```
{ echo; df -h /; } >> "$report"
```


3.5 Print the report path

- A successful script should tell the user what changed

Done message

```
echo "wrote $report" >&2
```

- Write status messages to stderr

Show path

```
printf 'Report: %s  
' "$report" >&2
```

3.6 Test report creation

- The first test is that the file exists

Exists test

```
[ -f system-report.txt ] || exit 1
```

- The second test is that the file is not empty

Not empty

```
[ -s system-report.txt ] || exit 1
```

3.7 Test report contents

- Search for labels that should always appear

Check hostname label

```
grep -q '^Hostname:' system-report.txt
```

- Avoid testing machine-specific exact values

Check disk label

```
grep -q '^Disk usage:' system-report.txt
```

4 Section 4

Project B backup-dir basics

4.1 Backup project requirements

- The script archives one directory

Requirement note

```
echo 'backup-dir DIR creates backups/*.tar.gz'
```

- Backups go into a backups folder

Default folder

```
backup_root=backups
```

4.2 Require one directory argument

- The input directory is required

Arg count

```
[ "$#" -eq 1 ] || exit 2
```

- Argument validation happens before work

Store arg

```
src=$1
```

4.3 Validate the source directory

- The backup target must be an existing directory

Directory check

```
[ -d "$src" ] || exit 1
```

- Use -d for directory tests

Error message

```
echo "not a directory: $src" >&2
```

4.4 Create the backups directory

- `mkdir -p` is safe when the directory already exists

Make output dir

```
mkdir -p "$backup_root"
```

- Keep generated archives out of the source tree

Writable output

```
[ -w "$backup_root" ] || exit 1
```


4.5 Use timestamped archive names

- Timestamps prevent overwriting older backups

Timestamp

```
stamp=$(date '+%Y%m%d-%H%M%S')
```

- Use sortable formats for filenames

Archive path

```
archive="backups/home-$stamp.tar.gz"
```

4.6 Use basename for readable names

- basename removes parent directories

Base name

```
name=$(basename "$src")
```

- The archive name can include the source folder name

Archive name

```
archive="backups/${name}-${stamp}.tar.gz"
```

4.7 Create tar.gz archives

- tar czf creates compressed gzip archives

Create archive

```
tar czf "$archive" "$src"
```

- The archive path comes before the source path

Verbose archive

```
tar czvf "$archive" "$src"
```

5 Section 5

Project B backup-dir polish

5.1 Print archive size

- Users want confirmation after backups

Archive size

```
du -h "$archive"
```

- du -h gives a readable size

Size message

```
echo "size: $(du -h "$archive" | cut -f1)" >&2
```

5.2 Add dry-run mode

- Dry-run mode shows what would happen

Dry flag

```
dry_run=1
```

- It is important for scripts that create large files

Dry message

```
[ "$dry_run" = 1 ] && echo "would tar $src"
```

5.3 Route actions through run

- A run helper centralizes dry-run behavior

Run helper

```
run() { echo "+ $" ">&2; "$@"; }
```

- It also makes verbose logging easier

Use run

```
run tar czf "$archive" "$src"
```

5.4 Keep the five newest backups

- Retention prevents backup folders from growing forever

Keep count

```
keep=5
```

- Start with a simple keep count

List newest

```
ls -lt backups/*.tar.gz | head -n "$keep"
```


5.5 Delete older backups carefully

- Only delete files matching this script's pattern

Old list

```
ls -lt backups/*.tar.gz | tail -n +6
```

- Review the list before wiring rm

Delete old

```
ls -lt backups/*.tar.gz | tail -n +6 | xargs -r rm
```

5.6 Handle tar failures

- A failed archive should stop the script

Tar with die

```
tar czf "$archive" "$src" || exit 1
```

- The error should name the source directory

Specific error

```
echo "backup failed: $src" >&2
```

5.7 Test restore manually

- A backup is only useful if it restores

Extract test

```
tar xzf "$archive" -C /tmp/restore-test
```

- Extract into a temporary directory

List restore

```
find /tmp/restore-test -maxdepth 2 -type f
```

6 Section 6

Project C log scanner basics

6.1 Log scanner requirements

- The script reads one log file

Requirement note

```
echo 'log-scan FILE prints counts'
```

- It counts total lines and severity levels

Input variable

```
log_file=$1
```

6.2 Create sample.log

- A sample file makes development repeatable

Sample line

```
echo 'INFO started' > sample.log
```

- Include INFO, WARN, and ERROR lines

More sample

```
echo 'ERROR failed' >> sample.log
```

6.3 Validate the log file

- The scanner needs a readable file

Arg count

```
[ "$#" -eq 1 ] || exit 2
```

- Use -f before counting

File check

```
[ -f "$log_file" ] || exit 1
```

6.4 Count total lines

- `wc -l` gives the total number of lines

Line count

```
total=$(wc -l < "$log_file")
```

- Redirect input to avoid printing the filename

Print total

```
echo "Total lines: $total"
```


6.5 Count ERROR lines

- `grep -c` counts matching lines

Error count

```
errors=$(grep -c 'ERROR' "$log_file" || true)
```

- ERROR count is usually the headline metric

Print errors

```
echo "ERROR: $errors"
```

6.6 Count WARN lines

- WARN lines often indicate risk without failure

Warn count

```
warns=$(grep -c 'WARN' "$log_file" || true)
```

- Use the same pattern as ERROR counts

Print warns

```
echo "WARN: $warns"
```

6.7 Count INFO lines

- INFO counts help compare normal activity

Info count

```
infos=$(grep -c 'INFO' "$log_file" || true)
```

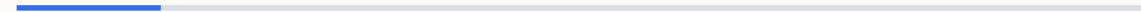
- The scanner should report all required severities

Print infos

```
echo "INFO: $infos"
```

7 Section 7

Project C log scanner details



7.1 Understand grep -c caveats

- grep returns 1 when there are no matches

No match status

```
grep -c ERROR clean.log  
echo $?
```

- That is not a scanner crash

Safe count

```
count=$(grep -c ERROR clean.log || true)
```

7.2 Show the last errors

- Recent errors are more useful than every error

Last errors

```
grep 'ERROR' "$log_file" | tail -n 5
```

- Use tail after grep

Allow none

```
grep 'ERROR' "$log_file" | tail -n 5 || true
```

7.3 Use pipefail with last errors

- pipefail may make no matches fail the pipeline

Pipeline

```
set -o pipefail  
grep ERROR "$log_file" | tail -n 5
```

- Decide whether no errors is acceptable

Expected none

```
grep ERROR "$log_file" | tail -n 5 || true
```

7.4 Format a scanner report

- Use labels that are easy to scan

Report header

```
echo 'Log Scan Report'
```

- Group counts before sample lines

Count line

```
printf 'ERROR: %s  
' "$errors"
```


7.5 Add a quiet machine-readable mode

- Some callers want just the numbers

CSV line

```
printf '%s,%s,%s\n' "$errors" "$warns" "$infos"
```

- A mode flag can switch output style

Mode variable

```
format=csv
```

7.6 Test log counts

- A sample log gives exact expected counts

Run scanner

```
output=$(./log-scan sample.log)
```

- Tests should fail when a count changes

Check count

```
grep -q 'ERROR: 1' <<< "$output"
```

7.7 Test no-error logs

- No-error input is a key edge case

Clean sample

```
echo 'INFO ok' > clean.log
```

- The script should still exit successfully

Clean test

```
./log-scan clean.log | grep -q 'ERROR: 0'
```

8 Section 8

Project D toolbelt CLI basics

8.1 Toolbelt project requirements

- A toolbelt script groups small utilities

Requirement note

```
echo 'toolbelt SUBCOMMAND [ARGS]'
```

- Subcommands choose the action

Example commands

```
toolbelt sysinfo  
toolbelt log-scan sample.log
```

8.2 Read the subcommand

- The first argument selects behavior

Read command

```
cmd="${1:-}"
```

- Use a default so missing commands are safe

Shift command

```
[ "$#" -gt 0 ] && shift
```

8.3 Write a top-level help message

- Help should list available subcommands

Help function

```
usage() { echo 'usage: toolbelt COMMAND'; }
```

- Print help for no arguments

No command

```
[ -n "$cmd" ] || { usage; exit 2; }
```

8.4 Use functions for subcommands

- Each subcommand belongs in one function

Function

```
cmd_sysinfo() { ./sysinfo-report; }
```

- Functions keep the case router short

With args

```
cmd_backup() { ./backup-dir "$@"; }
```


8.5 Route with case

- case is the standard Bash router for commands

Case route

```
case "$cmd" in
  sysinfo) cmd_sysinfo "$@";;
esac
```

- Each pattern calls one function

Default route

```
*) usage; exit 2;;
```

8.6 Return subcommand statuses

- The toolbelt should fail when the subcommand fails

Return status

```
cmd_backup() { ./backup-dir "$@"; }
```

- Do not hide the status with a final echo

Exit status

```
cmd_backup "$@"  
exit $?
```

8.7 Use exit 2 for unknown commands

- Unknown command is a usage problem

Unknown command

```
echo "unknown command: $cmd" >&2
```

- Show help so the user can recover

Usage exit

```
usage  
exit 2
```

9 Section 9

Project D toolbelt polish

9.1 Pass remaining arguments safely

- \$@ preserves argument boundaries when quoted

Safe pass

```
cmd_log_scan "$@"
```

- Subcommands should receive their own inputs

Function wrapper

```
cmd_log_scan() { ./log-scan "$@"; }
```

9.2 Reuse shared helper functions

- usage, die, and need can be shared across commands

Die helper

```
die() { echo "$*" >&2; exit 1; }
```

- Keep helpers near the top of the file

Need helper

```
need() { command -v "$1" >/dev/null || exit 127; }
```

9.3 Add per-command help

- Subcommands often need their own usage text

Help branch

```
[ "${1:-}" = --help ] && usage_backup
```

- Support help before running real work

Backup help

```
usage_backup() { echo 'usage: toolbelt backup DIR'; }
```

9.4 Package scripts in bin

- A bin directory is familiar for command-line tools

Bin layout

```
mkdir -p bin
```

- Keep executable entry points there

Move script

```
mv toolbelt bin/toolbelt
```


9.5 Install locally through PATH

- PATH lets users run the tool from any directory

Make local bin

```
mkdir -p "$HOME/bin"
```

- A local bin directory avoids system installs

Copy tool

```
cp bin/toolbelt "$HOME/bin/toolbelt"
```

9.6 Keep paths relative to the script

- The caller may run the tool from any directory

Script dir

```
script_dir=$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)
```

- Find the script directory with BASH_SOURCE

Sibling script

```
"$script_dir/backup-dir" "$@"
```

9.7 Run demo commands

- Demo commands prove the CLI is understandable

Help demo

```
bin/toolbelt --help
```

- Use examples that a reviewer can copy

Project demo

```
bin/toolbelt sysinfo
```

10

Section 10

Documentation and safety

10.1 Write a README for scripts

- A README explains what the project does

README title

```
echo '# Bash Toolbelt' > README.md
```

- Include requirements, usage, and examples

Usage line

```
echo 'Usage: bin/toolbelt COMMAND' >> README.md
```

10.2 Document dependencies

- Projects often rely on external commands

README dependency

```
echo 'Requires: bash tar grep' >> README.md
```

- List them so users can prepare

Runtime check

```
command -v tar >/dev/null || exit 127
```

10.3 Document exit codes

- Exit codes help automation callers

Exit docs

```
echo '0 success, 1 failure, 2 usage' >> README.md
```

- Explain success, usage errors, and runtime failures

Usage exit

```
exit 2
```

10.4 Use relative path safety

- Scripts should not depend on the caller's directory

Project root

```
root=$(cd "$(dirname "$0")/.." && pwd)
```

- Resolve paths before using sibling files

Sample path

```
sample="$root/samples/sample.log"
```


10.5 Prefer BASH_SOURCE for sourced paths

- BASH_SOURCE points at the current script file

Dirname pattern

```
dir=$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)
```

- \$0 can point at the parent command when sourced

Project root

```
root=$(cd "$dir/.." && pwd)
```

10.6 Keep generated files out of source

- Generated reports and archives can clutter commits

Ignore reports

```
echo '*.tar.gz' >> .gitignore
```

- Use output directories and ignore patterns

Check status

```
git status --short
```

10.7 Show safe destructive commands

- Any rm command needs extra care

Preview delete

```
printf 'delete %s  
' "$old_file"
```

- Print what will be removed before deleting

Guard delete

```
[ "$dry_run" = 1 ] || rm -- "$old_file"
```

11

Section 11

Build plan and acceptance

11.1 Build one vertical slice

- A vertical slice runs end to end with minimal features

Tiny slice

```
echo 'System Report' > report.txt
```

- It proves the script shape early

Run slice

```
./sysinfo-report
```

11.2 Add one feature at a time

- Small changes make bugs easier to find

Add hostname

```
echo "Hostname: $(hostname)" >> report.txt
```

- Run the script after each feature

Run again

```
./sysinfo-report && grep Hostname report.txt
```

11.3 Keep an acceptance checklist

- Acceptance checks define done

Checklist item

```
echo '[ ] report file is created'
```

- They should be observable from the command line

Check item

```
[ -s system-report.txt ] && echo pass
```

11.4 Create a manual test checklist

- Manual checks are useful for beginner projects

Success test

```
./backup-dir samples
```

- Include success and failure cases

Failure test

```
./backup-dir missing || echo expected
```


11.5 Automate the easiest checks

- Turn repeated manual checks into scripts

Test script

```
bash tests/sysinfo_test.sh
```

- Start with file existence and output labels

Simple assert

```
grep -q Hostname system-report.txt
```

11.6 Run syntax checks for every script

- Syntax checks are fast and low effort

Bash syntax

```
bash -n bin/toolbelt
```

- They catch broken edits before demos

All bin scripts

```
for f in bin/*; do bash -n "$f"; done
```

11.7 Prepare demo runs

- A demo script removes typing pressure

Demo script

```
echo './bin/toolbelt sysinfo' > demo.sh
```

- Use fresh sample inputs

Run demo

```
bash demo.sh
```

1 2

Section 12

Common bugs and portfolio polish

12.1 Common bug: unquoted project paths

- Project demos often use simple paths first

Space path

```
mkdir -p 'sample dir'
```

- Real users have spaces in directories

Quoted run

```
./backup-dir 'sample dir'
```

12.2 Common bug: missing execute bit

- Permission errors confuse first-time users

Check mode

```
test -x bin/toolbelt
```

- Check executable mode before demos

Fix mode

```
chmod +x bin/toolbelt
```

12.3 Common bug: outputs overwritten

- Reports and backups can overwrite earlier runs

Timestamp file

```
file="report-$(date +%Y%m%d).txt"
```

- Use timestamps when history matters

Fixed latest

```
latest=system-report.txt
```

12.4 Stretch goal: JSON-like output

- Machine-readable output makes scripts easier to integrate

Simple JSON line

```
printf '{"errors":%s}'  
' "$errors"
```

- Keep the first version simple

Plain fallback

```
printf 'ERROR: %s'  
' "$errors"
```


12.5 Stretch goal: configuration file

- A config file avoids long command lines

Source config

```
./toolbelt.conf
```

- Source only trusted config files

Default value

```
keep="${keep:-5}"
```

12.6 Portfolio tip: include screenshots or transcripts

- A transcript shows the tool working

Save transcript

```
script -q demo.txt ./demo.sh
```

- Keep sample output short and readable

Show output

```
sed -n '1,20p' demo.txt
```

12.7 Final project walkthrough

- Walk through requirements, code, tests, and demo

Walkthrough order

```
echo 'requirements code tests demo'
```

- Explain one bug you fixed

Next step

```
echo 'Next: add automated retention tests'
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory
08 · Projects

